SC2006 SOFTWARE ENGINEERING · LAB 5

Crowdy

Check how crowded Singapore's malls and MRT stations are before you go, and find a fair place for friends to meet.

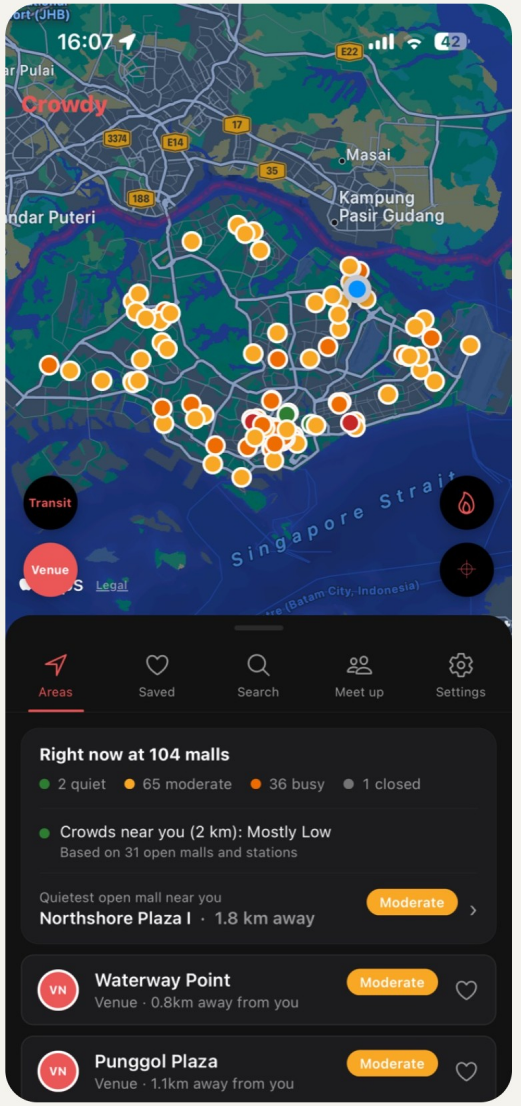
SCSJ Group 3 · Tan Leong Jun Joseph · Teo Kok Wei, Oscar · Mao Yikai · Xu Hanwen · K Eswaran

THE PROBLEM

You only find out a place is packed when you get there.

Crowdy shows how busy 104 malls and every MRT/LRT station are before you leave, points you to quieter malls nearby, and helps a group of friends choose a mall that is fair for everyone to reach.

For commuters, students and residents in Singapore. Works as a guest, with an account for synced saves and friends.



31 use cases in nine groups

1 Background crowd data

2 Map and nearby venues

3 Venue detail

4 Crowd heatmap

5 Search

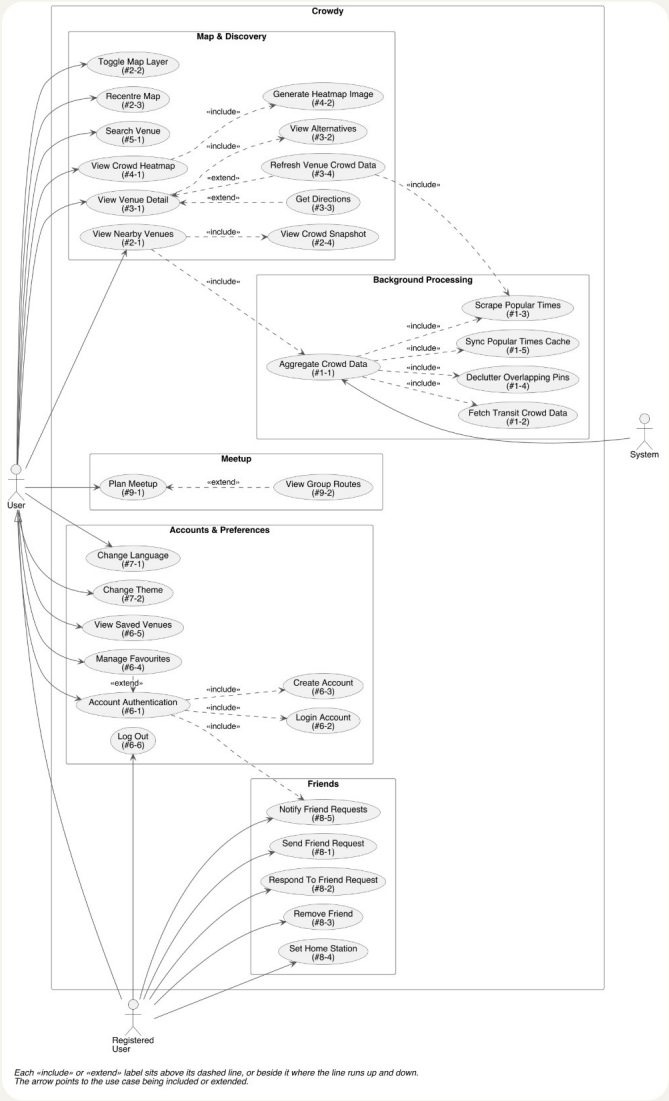
6 Accounts and saved venues

7 Preferences

8 Friends

9 Meetup planner

Three actors: **User** (guest), **Registered User** (signed in) and the **System** (the backend on its own schedule).



Three free data sources, combined

LTA DataMall

Live crowd level at every MRT/LRT station, refreshed every 5 minutes. Carpark lots for malls.

Google Popular Times

Each mall's typical week, hour by hour, and its opening hours.

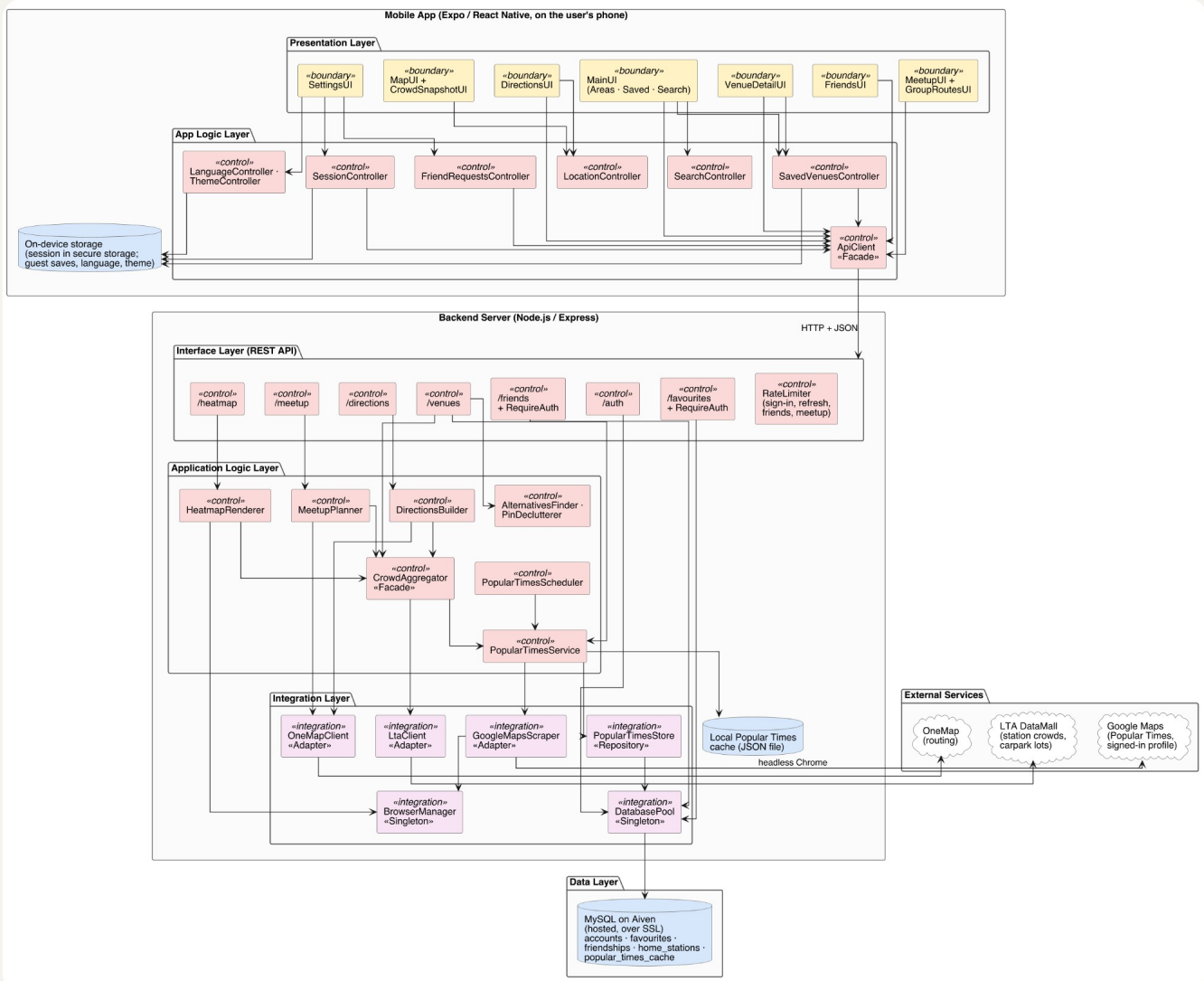
OneMap

Public transport, walking, driving and cycling routes, and the travel times behind the meetup planner.

None of them answers "where should we go right now?" on its own. Crowdy merges them into one venue list.

Demo

1. Home map and the Right now card
2. Selecting a venue
3. Venue page and quieter alternatives
4. Directions with live station crowds
5. Heatmap and search
6. Accounts, saved venues and security
7. Friends and the request alert
8. Meetup planner and group routes
9. English / 中文, dark and light



Layered architecture

App: screens (boundary) call controllers, and only ApiClient talks to the server.

Server: routes, then application logic, then adapters for LTA, Google, OneMap and MySQL.

Database: hosted MySQL on Aiven, plus a local Popular Times cache.

Software engineering principles

Separation of concerns

App and server are separate. Screens, controllers and entities are kept apart.

Low coupling, high cohesion

Screens only know ApiClient. Each adapter wraps one outside service.

Information hiding

PopularTimesStore hides the SQL. OneMapClient keeps its login token to itself.

Abstraction

One Venue type for malls and stations, so every screen treats them alike.

Don’t repeat yourself

One strings file for both languages. One step row shared by directions and group routes.

Fail-safe design

If a source fails, the last good data is kept. A local cache covers a database outage.

SOLID in Crowdy

Principle	Where it shows
Single responsibility	Each server service owns one concern: lta.ts talks to LTA, popularTimes.ts handles Popular Times, meetup.ts ranks malls
Open/closed	Heatmap, friends and meetup were added as new modules, without editing the crowd data code
Liskov substitution	A mall from Google and a station from LTA are both Venues, and either can be used wherever a Venue is expected
Interface segregation	ApiClient offers small, specific functions (getFavourites, sendFriendRequest) instead of one general request call
Dependency inversion	Routes call service functions such as getVenues(), not Puppeteer or LTA directly, so a data source can change underneath

Design patterns

Pattern	Where	Why
Facade	ApiClient, CrowdAggregator	One entry point over many calls and sources
Adapter	LtaClient, GoogleMapsScraper, OneMapClient	Turn each outside service into Crowdy’s own types
Singleton	BrowserManager, DatabasePool	One shared browser and one connection pool
Repository	PopularTimesStore	Hides the database behind save and load
Observer	Session, Language, Theme, FriendRequests	Screens update when shared state changes

Each pattern is marked on the class diagrams in SRS Appendix B.

Design choices we made

A list that holds still

Found in testing: the list reshuffled as the map moved. It now freezes while a venue is selected.

Ask before merging

Guest saves are added to an account only after asking, fixing a leak between accounts on one phone.

OneMap over Google

Free, no billing account, authoritative for Singapore.

In-app alerts

Friend requests alert on sign-in and on return to the app, with no push service needed.

Fast meetup search

Shortlist by distance, at most 8 lookups at once, trips cached for 6 hours.

Testing and quality

48
server tests, all passing

24
app tests, all passing

0
critical findings in the Scantist scan

- Black-box tests (equivalence classes, boundary values) and white-box basis path tests from Lab 4
- Security: bcrypt passwords, signed session tokens, rate limiting per client
- JSDoc on key classes and public methods

TRACEABILITY 1

ViewAlternatives (#3-2)

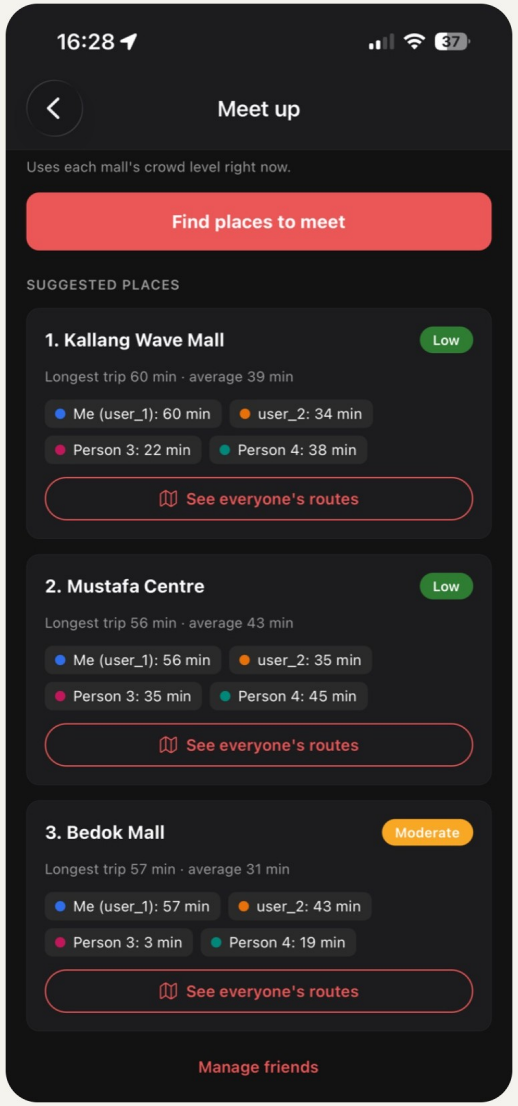
Requirement	Use case	Design	Code	Tests
REQ-3.6: up to 3 quieter malls within 3 km, nearest first	#3-2: same category, within 3 km, lower crowd, sorted by distance	AlternativesFinder, called by VenueController (Figure B.4)	server/src/services/alternatives.ts, findAlternatives	Lab 4 black-box tests: BB-10 at 3.000 km is in, BB-11 at 3.001 km is out

One requirement, followed from the SRS to the use case, the class diagram, the code and the tests.

TRACEABILITY 2

PlanMeetup (#9-1)

Step	Where
Requirements	REQ-9.1 to REQ-9.10
Use case	#9-1, with later time, friend without a home station and all-closed flows
Design	MeetupUI (B.3), MeetupController and MeetupPlanner (B.4), OneMapClient (B.5)
Code	server/src/services/meetup.ts: planMeetup, rankMeetupOptions
Tests	server/tests/meetup.test.ts



Future work

Sign in with Google

Replace the username and password with each user's own Google account.

Push notifications

Tell users about friend requests even when the app is closed.

Android and Web maps

A web-based map, for example Leaflet in a WebView, since Expo Go dropped Google Maps.

Licensed crowd data

Replace the Popular Times collection with a licensed source if one becomes available.

QnA

The SRS, use case model, diagrams and source code are in our repository under deliverables/Lab 5 Deliverables.

SCSJ Group 3 · Crowdy